



本小节内容

堆排序实战

堆排序实战

上一小节我们讲解了堆排序的原理，下面我们来代码实战。

代码实战步骤：首先我们通过随机数生成 10 个元素，通过随机数生成，我们可以多次测试排序算法是否正确，然后打印随机生成后的元素顺序，然后通过堆排序对元素进行排序，然后再次打印排序后的元素顺序。

堆排序的步骤是首先把堆调整为大根堆，然后我们交换根部元素也就是 A[0]，和最后一个元素，这样最大的元素就放到了数组最后，接着我们将剩余 9 个元素继续调整为大根堆，然后交换 A[0]和 9 个元素的最后一个，循环往复，直到有序。

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
typedef int ElemType;
typedef struct{
    ElemType *elem;
    int TableLen;
}SSTable;

void ST_Init(SSTable &ST,int len)//申请空间，并进行随机数生成
{
    ST.TableLen=len;
    ST.elem=(ElemType *)malloc(sizeof(ElemType)*ST.TableLen);
    int i;
    srand(time(NULL));
    for(i=0;i<ST.TableLen;i++)
    {
        ST.elem[i]=rand()%100;
    }
}
void ST_print(SSTable ST)
{
    for(int i=0;i<ST.TableLen;i++)
```



```
{
    printf("%3d",ST.elem[i]);
}
printf("\n");
}
void swap(ElemType &a,ElemType &b)
{
    ElemType tmp;
    tmp=a;
    a=b;
    b=tmp;
}

//调整某个父亲节点，王道书上的
void AdjustDown(ElemType A[],int k,int len)
{
    int i;
    A[0]=A[k];
    for(i=2*k;i<=len;i*=2)
    {
        if(i<len&&A[i]<A[i+1])//左子节点与右子节点比较大小
            i++;
        if(A[0]>=A[i])
            break;
        else{
            A[k]=A[i];
            k=i;
        }
    }
    A[k]=A[0];
}

//用数组去表示树 类似层次建树 王道书上的
void BuildMaxHeap(ElemType A[],int len)
{
    for(int i=len/2;i>0;i--)
    {
        AdjustDown(A,i,len);
    }
}

//王道书上的
void HeapSort(ElemType A[],int len)
{
    int i;
    BuildMaxHeap(A,len);//建立大顶堆
```



```
for(i=len;i>1;i--){
    swap(A[i],A[1]);
    AdjustDown(A,1,i-1);
}
//调整子树
void AdjustDown1(ElemType A[], int k, int len)
{
    int dad = k;
    int son = 2 * dad + 1; //左孩子下标
    while (son<=len)
    {
        if (son + 1 <= len && A[son] < A[son + 1])//看下有没有右孩子，比较左右孩子选大的
        {
            son++;
        }
        if (A[son] > A[dad])//比较孩子和父亲，如果孩子大于父亲，那么进行交换
        {
            swap(A[son], A[dad]);
            dad = son;//孩子重新作为父亲，判断下一颗子树是否符合大根堆
            son = 2 * dad + 1;
        }
        else {
            break;
        }
    }
}
void HeapSort1(ElemType A[], int len)
{
    int i;
    //建立大根堆
    for (i = len / 2; i >= 0; i--)
    {
        AdjustDown1(A, i, len);
    }
    swap(A[0], A[len]);//交换顶部和数组最后一个元素
    //下面的策略是，不断调整剩余元素为大根堆，因为根部最大，所以再次与 A[i]交换（相当于放到数组后面），循环往复
    for (i = len - 1; i > 0; i--)
    {
        AdjustDown1(A, 0, i);//剩下元素调整为大根堆
        swap(A[0], A[i]);
    }
}
```



```
}
```

```
//《王道C督学营》课程
```

```
//堆排序
```

```
int main()
```

```
{
```

```
    SSTable ST;
```

```
    ST_Init(ST,10);//初始化
```

```
    ElemType A[10] = { 3, 87, 2, 93, 78, 56, 61, 38, 12, 40 };
```

```
    memcpy(ST.elem,A,sizeof(A));
```

```
    ST_print(ST);
```

```
    //HeapSort(ST.elem, 9);//王道书零号元素不参与排序，考研考的都是零号元素要参与排序
```

```
    HeapSort1(ST.elem,9);//所有元素参与排序
```

```
    ST_print(ST);
```

```
    return 0;
```

```
}
```

AdjustDown1 函数的循环次数是 $\log_2 n$ ，HeapSort1 函数的第一个 for 循环了 $n/2$ 次，第二个 for 循环了 n 次，总计次数是 $3/2 n \log_2 n$ 次，因此时间复杂度是 $O(n \log_2 n)$ 。

堆排最好、最坏、平均时间复杂度都是 $O(n \log_2 n)$ 。

堆排的空间复杂度是 $O(1)$ ，因为没有使用与 n 相关的额外空间。